



รายงาน

แอปสั่งอาหารสำหรับลูกค้าที่นั่งในบ้าน

เสนอ

อ.ดร.วรัทภพ ฐภัทรสุวรรณ

จัดทำโดย

นางสาวปริญดา เสนา	รหัสนิสิต 6721601338
นางสาววิภาดา ทับเปี้ย	รหัสนิสิต 6721601460
นายศุภวิชญ์ พุ่มพฤษ	รหัสนิสิต 6721601508
นางสาวอภิญรัฐชญา หวังแก้ว	รหัสนิสิต 6721601591

รายงานฉบับนี้เป็นส่วนหนึ่งของการศึกษาในรายวิชา
การออกแบบและพัฒนาโปรแกรมประยุกต์สำหรับอุปกรณ์เคลื่อนที่
รหัสวิชา 01418342

ภาคต้น ปีการศึกษา 2569 คณะศิลปศาสตร์และวิทยาศาสตร์

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

1. แผนผังฐานข้อมูล วาดตารางทั้งหมดพร้อมเส้นความสัมพันธ์ ระบุคีย์หลักและคีย์นอก

1. Category เก็บข้อมูลหมวดหมู่อาหาร

ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
category_id	รหัสหมวดหมู่	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
category_name	ชื่อหมวดหมู่	TEXT	-	NOT NULL	-

2.Menu เก็บข้อมูลเมนูอาหารและราคาปัจจุบัน

ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
menu_id	รหัสเมนู	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
name	ชื่อเมนู	TEXT	-	NOT NULL	-
price	ราคา	INTEGER	-	NOT NULL, CHECK (price >= 0)	-
category_id	รหัสหมวดหมู่	INTEGER	FOREIGN KEY	NOT NULL REFERENCES categories (categories_id)	ON DELETE RESTRICT ป้องกันการลบหมวดหมู่อาหารหากยังมีรายการอาหารในหมวดนั้นอยู่

3.Table เก็บข้อมูลโต๊ะในร้าน

ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
table_id	รหัสโต๊ะ	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
table_number	เลขโต๊ะ	TEXT	-	NOT NULL	-
table_status	ราคา	INTEGER	-	NOT NULL, DEFAULT 'available', CHECK (status IN ('available', 'unavailable'))	-

4.Bills เก็บข้อมูลบิลร้าน

ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
bills_id	รหัสบิล	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
table_id	รหัสโต๊ะ	INTEGER	FOREIGN KEY	NOT NULL, REFERENCES tables(table_id)	ON DELETE RESTRICT ป้องกันการลบข้อมูลโต๊ะออกจากระบบ หากโต๊ะนั้นเคยมีประวัติการเปิดบิล
open_at	เวลาเปิดบิล	TEXT	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP	-

close_at	เวลาปิดบิล	TEXT	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP	-
bills_status	สถานะ	TEXT	-	NOT NULL DEFAULT 'open', CHECK (status IN ('open', 'close','cancel'))	-

5.Order เก็บข้อมูลการสั่งซื้อ

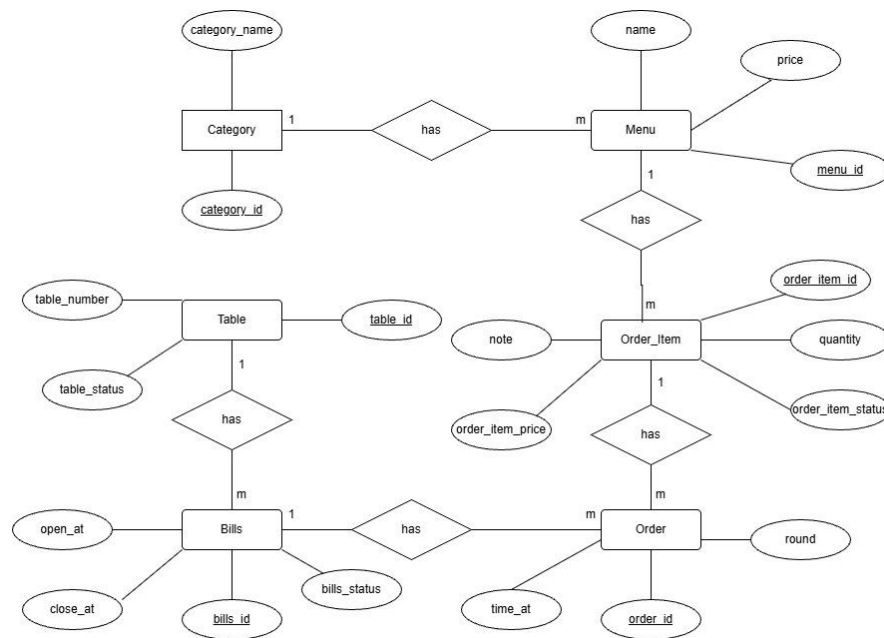
ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
order_id	รหัสการสั่งซื้อ	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
bills_id	รหัสโต๊ะ	INTEGER	FOREIGN KEY	NOT NULL, REFERENCES bills(bills_id)	ON DELETE CASCADE หากบิลถูกลบออก เช่นถูกยกเลิก การจะลบข้อมูลที่ผูกกับบิลทั้งหมด ออกไปด้วย ข้อมูลจะได้ไม่ตกค้าง
round	รอบที่สั่งซื้อ	TEXT	-	NOT NULL, CHECK (round > 0)	-
time_at	เวลาที่สั่งซื้อ	TEXT	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP	-

6.Order_Item เก็บข้อมูลรายการสั่งซื้ออาหารในแต่ละรอบ

ชื่อ Attribute	คำอธิบาย	ชนิดข้อมูล	ประเภท Key	เงื่อนไข	พฤติกรรมตอนลบ
order_item_id	รหัสรายการ	INTEGER	PRIMARY KEY	AUTOINCREMENT	-
order_id	รหัสการสั่ง	INTEGER	FOREIGN KEY	NOT NULL, REFERENCES order(order_id)	ON DELETE CASCADE หากรอบการสั่งถูกลบ รายการอาหารภายในรอบนั้นต้องถูกลบตามทันที
menu_id	รหัสเมนู	INTEGER	-	NOT NULL, REFERENCES menu(menu_id)	ON DELETE RESTRICT ป้องกันการลบรายการอาหารออกจาก ฐานข้อมูล หากรายการนั้นเคยถูกสั่งในออเดอร์ ที่ย้อนหลังไปแล้ว
quantity	จำนวนที่สั่ง	INTEGER	-	NOT NULL CHECK (quantity >= 0)	-
note	หมายเหตุ	TEXT	-	NULL	-
order_item_status	สถานะการทำอาหาร	TEXT	-	NOT NULL DEFAULT(pending) CHECK(status IN('pending',	-

				'cooking , 'served', 'cancelled'))	
order_item_price	ราคา ณ ตอนสั่ง (ล็อกราคาให้คงเดิมไว้ตลอด)	INTEGER	-	NOT NULL, CHECK (order_item_price >= 0)	-

ER Diagram:



2. เหตุผลของการแยกตาราง อธิบายว่าทำไมข้อมูลชุดนั้นจึงต้องอยู่คนละตาราง แทนที่จะรวมไว้ตารางเดียว

1. Category ออกจาก Menu

- o ลดซ้ำซ้อน: ไม่ต้องพิมพ์ชื่อหมวดหมู่ซ้ำในทุกเมนู
- o อัปเดตง่าย: แก้ไขชื่อหมวดหมู่ที่ตาราง **Category** ที่เดียว
- o ลดผิดพลาด: ป้องกันการสะกดชื่อหมวดหมู่ผิด

2. Table ออกจาก Bills

- o รองรับ One-to-Many: 1 โต๊ะมีประวัติการเปิดบิลได้หลายครั้ง
- o แยกสถานะกับประวัติ: **tables** เก็บสถานะปัจจุบัน (ว่าง/ไม่ว่าง) ส่วน **bills** เก็บประวัติ ยอดเงิน และเวลา

3. Bills และ Orders

- o **รองรับสั่งหลายรอบ:** ระบุเวลาและลำดับรอบของออเดอร์ในบิลนั้น ๆ ได้ถูกต้อง
- o **คุมครัวง่าย:** แยกรายการอาหารตามรอบที่สั่งจริงได้ชัดเจน

4. Orders และ Order_Item

- o **รองรับ Many-to-Many:** 1 ออเดอร์มีได้หลายเมนู และ 1 เมนูอยู่ได้หลายออเดอร์
- o **เก็บข้อมูลเฉพาะรายการ:** บันทึกจำนวน เหตุผล (เช่น เพ็ดน้อย) และสถานะแยกรายการได้

3. การเก็บราคา อธิบายวิธีที่ใช้แก้ปัญหาการเปลี่ยนย้อนหลัง และเหตุผลที่เลือกวิธีนั้น

- **วิธีแก้:** เพิ่มคอลัมน์ `order_item_price` ในตาราง `order_items`
- **เหตุผล:** ล็อกราคา ณ วินาทีที่สั่ง ป้องกันไม่ให้เกิดการปรับราคาในตาราง `menu` ส่งผลให้ยอดขายในบิลเก่าผิดพลาด

4. การรองรับหลายรอบการสั่ง อธิบายว่าออกแบบอย่างไรให้รู้ว่ารายการใดอยู่รอบไหน และทำไมต้องรู้

- **การออกแบบ:** บันทึก `Timestamp (created_at)` และลำดับรอบ (`order_round`) ไว้ในตาราง `orders`
- **ทำไมต้องรู้:** ช่วยครัวจัดคิวทำอาหารก่อน-หลัง และช่วยระบบคำนวณยอดเงินรวมได้ครบถ้วน

5. คำสั่ง SQL สามคำสั่งที่ซับซ้อนที่สุดในงาน ยกมาพร้อมอธิบายที่ละส่วนว่าแต่ละบรรทัดทำอะไร

1. `getOrderItems()`: ใช้ `JOIN` เชื่อม 5 ตาราง ได้แก่ `order_item`, `menu`, `orders`, `bills` และ `tables` เพื่อดึงข้อมูลว่าโต๊ะไหนสั่งเมนูอะไร จำนวนเท่าไร ราคาทั้งหมดเท่าไร
2. `getSalesSummary()`: ใช้ `COUNT`, `SUM`, `COALESCE` และ `LEFT JOIN` เพื่อคำนวณจำนวนบิลที่ปิดแล้ว ยอดขายรวม และจำนวนอาหารที่ขายได้ โดย `COALESCE` ใช้เปลี่ยนค่า `NULL` ให้เป็น 0
3. `createOrder()`: ใช้ `Transaction` เพื่อให้การสร้าง `Order` เป็นชุดเดียวกัน โดยตรวจสอบโต๊ะ หา `Bill` ที่เปิดอยู่ ถ้าไม่มีจะสร้าง `Bill` ใหม่ จากนั้นหารอบถัดไป สร้าง `Order` เพิ่มรายการอาหารลง `order_item` และเปลี่ยนสถานะโต๊ะเป็น `unavailable` ถ้าขั้นตอนใดผิดพลาด `Transaction` ไม่ควรปล่อยให้ข้อมูลบางส่วนถูกบันทึกค้างไว้

6. ดัชนีบอกกว่าสร้างที่คอลัมน์ใด เพราะเหตุใด และแนบผล EXPLAIN QUERY PLAN

ที่แสดงว่าถูกใช้จริง `CREATE INDEX`

1.

`CREATE INDEX idx_bills_tables_status ON bills(tables_id, bills_status);`

o เหตุผล: ดึงข้อมูลปัจจุบันของโต๊ะได้เร็ว ไม่ต้องค้นหาจากประวัติบิลทั้งหมด

o ผล EXPLAIN: SEARCH TABLE bills USING INDEX
idx_bills_tables_status (tables_id=? AND bills_status=?)

2. CREATE INDEX idx_order_item_order_id ON order_item(order_id);

o เหตุผล: ดึงรายการอาหารของออเดอร์นั้น ๆ ได้ทันที ไม่ต้องอ่านข้อมูลทั้งหมด

o ผล EXPLAIN: SEARCH TABLE order_item USING INDEX
idx_order_item_order_id (order_id=?)

7. ทราบแซกชัน บอกว่าใช้ที่ใดบ้าง และอธิบายว่าถ้าไม่ใช้จะเกิดอะไรขึ้น

- จุดที่ใช้ (db.js):
 - o initDatabase (บรรทัด 92): การสร้างตาราง ลงข้อมูลตั้งต้น และสร้าง Index
 - o resetDatabase (บรรทัด 262): การ DROP TABLE ทั้งหมดก่อนสร้างใหม่
- ถ้าไม่ใช้: หากเกิด Error กลางคัน ข้อมูลจะค้างครึ่ง ๆ กลาง ๆ โครงสร้าง DB พัง และแอปใช้งานต่อไม่ได้

8. ปัญหาที่เจอและวิธีแก้อย่างน้อย 3 เรื่อง เขียนอาการที่เห็น สาเหตุที่แท้จริง และวิธีแก้ปัญหา

ปัญหา 1: เปิดแอปแล้วข้อมูลซ้ำ/Error

- สาเหตุ: Seed Data ทำงานทุกครั้งโดยไม่เช็คข้อมูลเดิม
- วิธีแก้: เปลี่ยนคำสั่ง SQL เป็น INSERT OR IGNORE INTO

ปัญหา 2: ลบบิล/โต๊ะแล้วติด FK Error หรือข้อมูลค้าง

- สาเหตุ: ตั้งค่า ON DELETE ไม่เหมาะสม
- วิธีแก้: ตั้งค่าความสัมพันธ์ bills -> orders -> order_item เป็น ON DELETE CASCADE ส่วน tables และ menu ใช้ ON DELETE RESTRICT

ปัญหา 3: ฝั่งครัวไม่อัปเดตเมื่อสั่งเพิ่ม

- สาเหตุ: ดึงข้อมูลผ่าน useEffect แค่ออนเปิดหน้าครั้งแรก
- วิธีแก้: ใช้ useEffect สั่งดึงข้อมูลใหม่ทุกครั้งที่สลับกลับมาหน้านั้น

9. การแบ่งงานใครทำอะไร พร้อมสัดส่วนโดยประมาณ

1. 6721601338 ปรีณดา เสนา : รับผิดชอบการออกแบบตาราง **database** เขียน **schema.sql** ข้อมูลตั้งต้น และฟังก์ชันอ่านเขียนพื้นฐาน เขียนรายงาน
2. 6721604160 วิภาดา ทับเปีย : รับผิดชอบ **UI** ฝั่งครัว หน้ารายการที่ต้องทำการเปลี่ยนสถานะ และหน้าสรุปยอดขาย
3. 6721601508 ศุภวิชญ์ พุ่มพฤษ : รวมโค้ด ทดสอบตามรายการตรวจสอบ เขียนรายงาน และตัดต่อวิดีโอ
4. 6721601591 อภินัฐชญา หวังแก้ว : รับผิดชอบ **UI** ฝั่งลูกค้า หน้าเลือกโต๊ะ หน้าเมนู ตะกร้า และหน้าสรุปบิล

10. สิ่งที่จะทำต่อถ้ามีเวลา อย่างน้อย 3 ข้อ

1. ระบบจัดการคิวและระบบย้ายโต๊ะ/รวมโต๊ะ (Table Management & Merge/Split Tables)
2. ระบบจัดการส่วนลด โปรโมชั่น และคูปองสะสมแต้ม (Discount & Loyalty Program)
3. ระบบรวมเงินแยกจ่ายตามคน / หารบิล (Split Bill / Bill Calculator)